

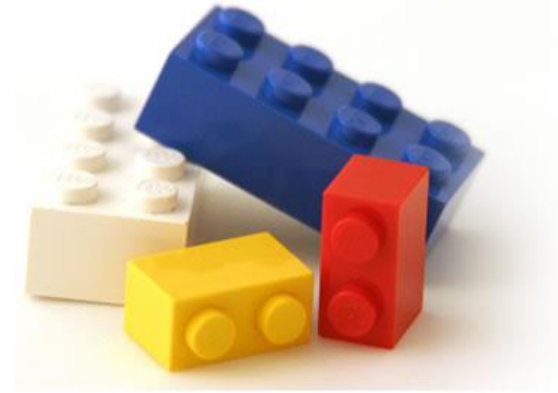
Unit Testing from Scratch

Prof. Dr. Raphael Herding

Chapter 1

The Basics

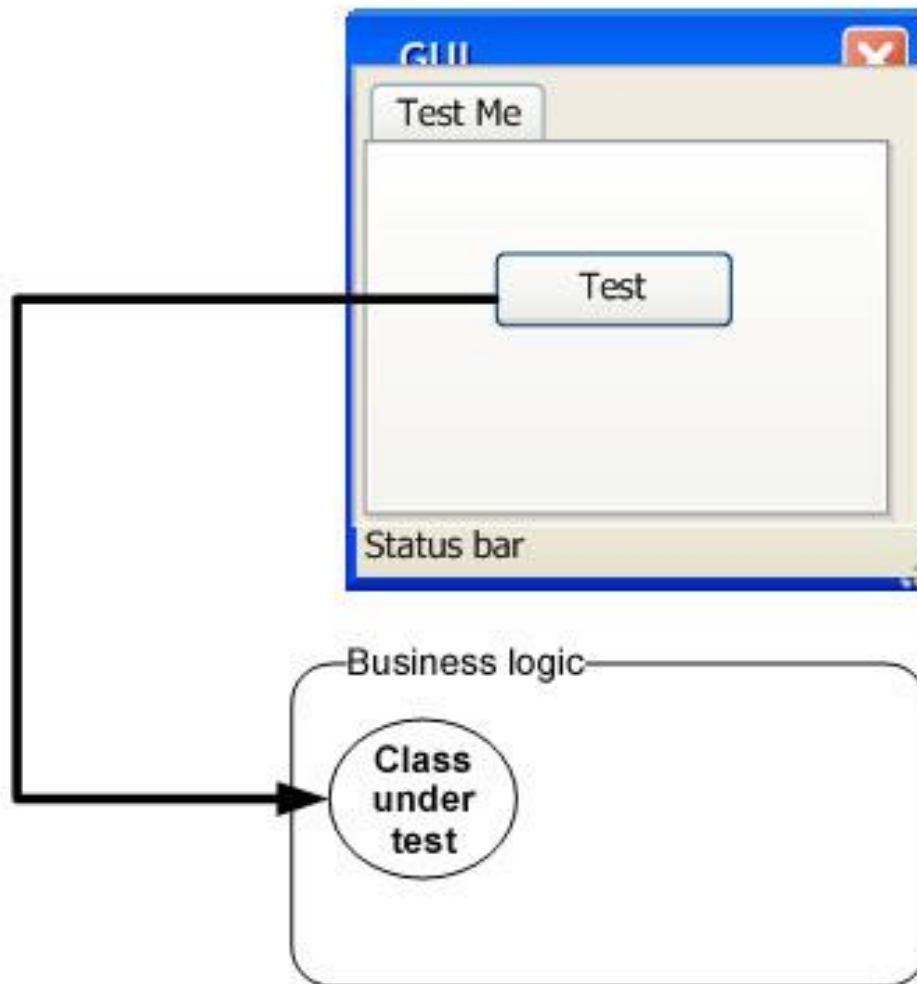
A “**unit**” is a method or function



A ***unit test*** is a piece of a code (usually a method) that invokes another piece of code and checks the correctness...



We've all Written Unit Tests (sort of)



A Unit Test *Should* have the Following Properties

1. It should be automated and repeatable.
2. It should be easy to implement.
3. Once it's written, it should remain for future use.
4. Anyone should be able to run it.
5. It should run at the push of a button.
6. It should run quickly.



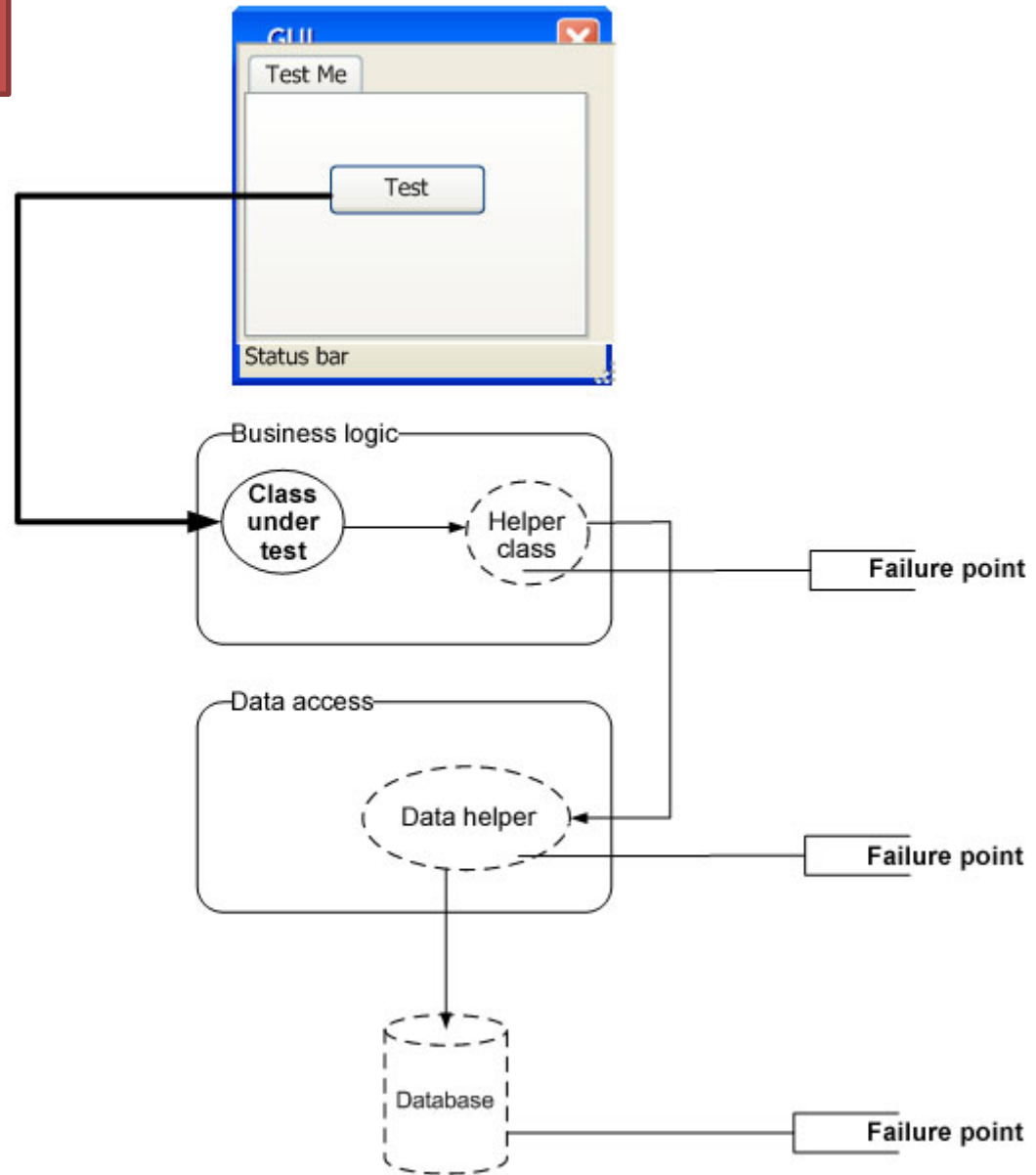


“What was I doing until now?”
you might ask.



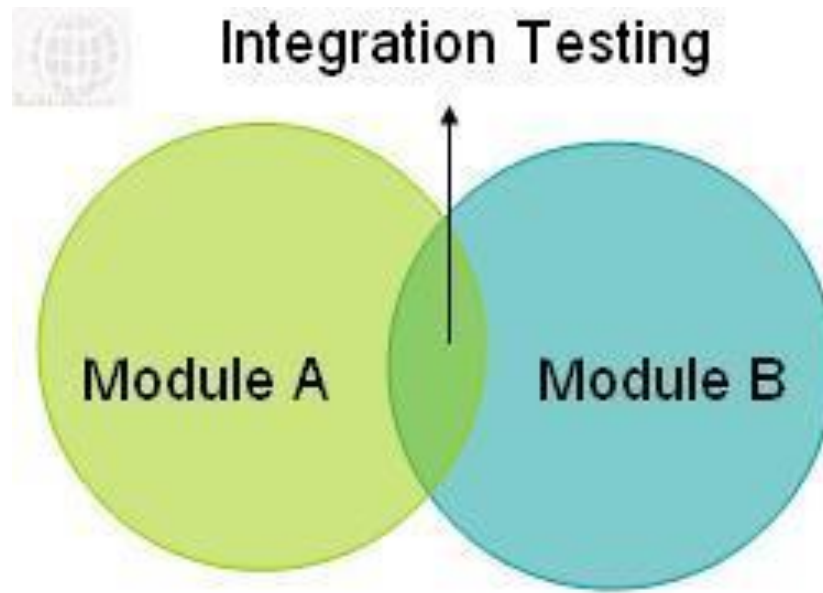
You’ve done *integration testing*.

integration test

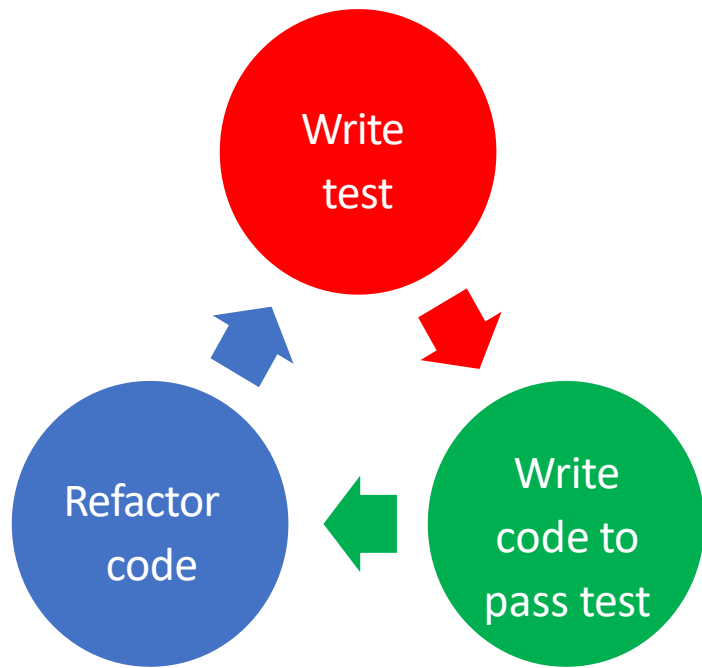


DEFINITION

- *Integration testing* means testing two or more dependent software modules as a group.

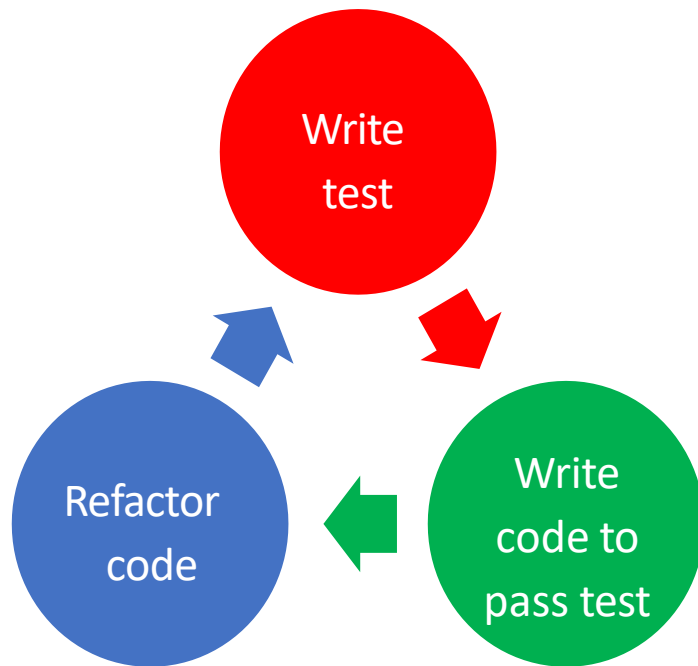


Test-driven Development

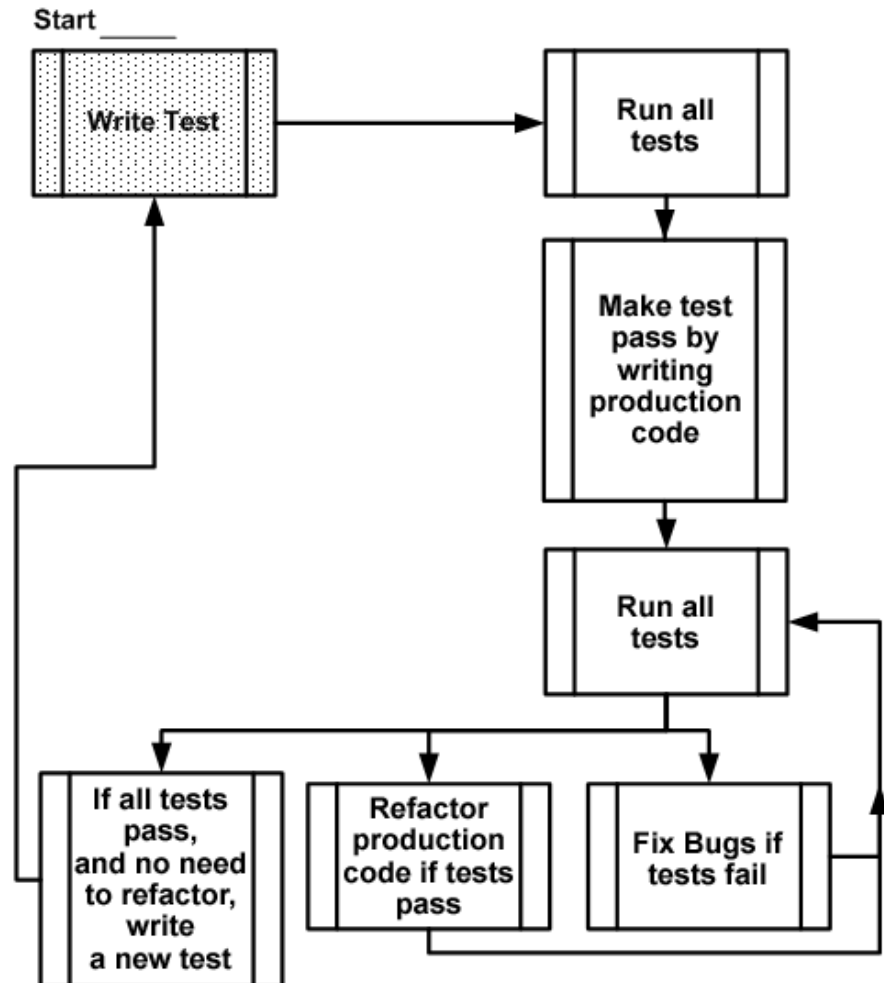


Red green refactoring

Test-driven Development

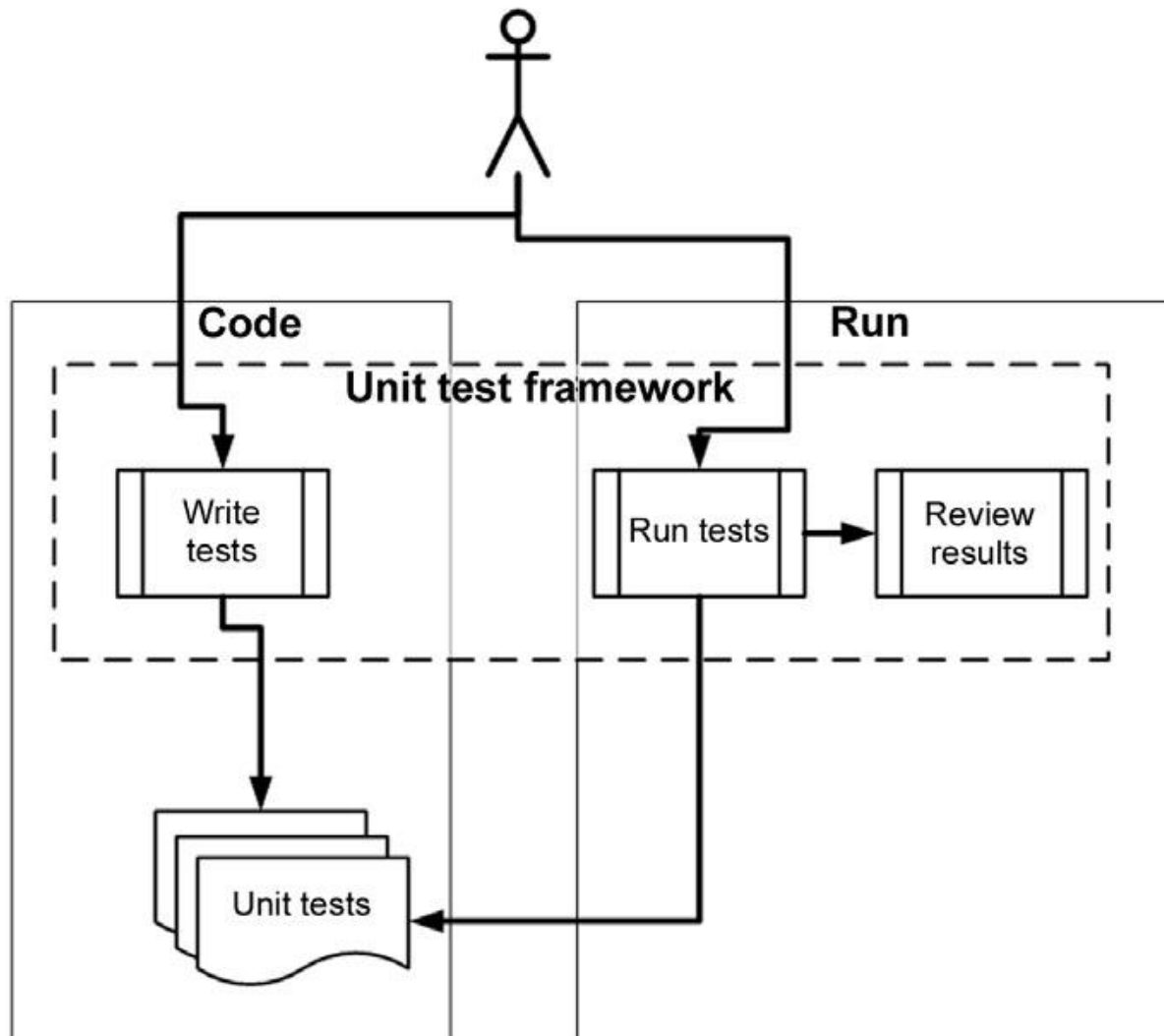


Red green refactoring



Chapter 2:

A First Unit Test



Log and Notification Project

```
public class LogAnalyzer
{
    public bool IsValidLogFileName(string fileName)
    {
        if(!fileName.EndsWith(".SLF"))
        {
            return true;
        }
    }
}
```

Rules for Naming Tests

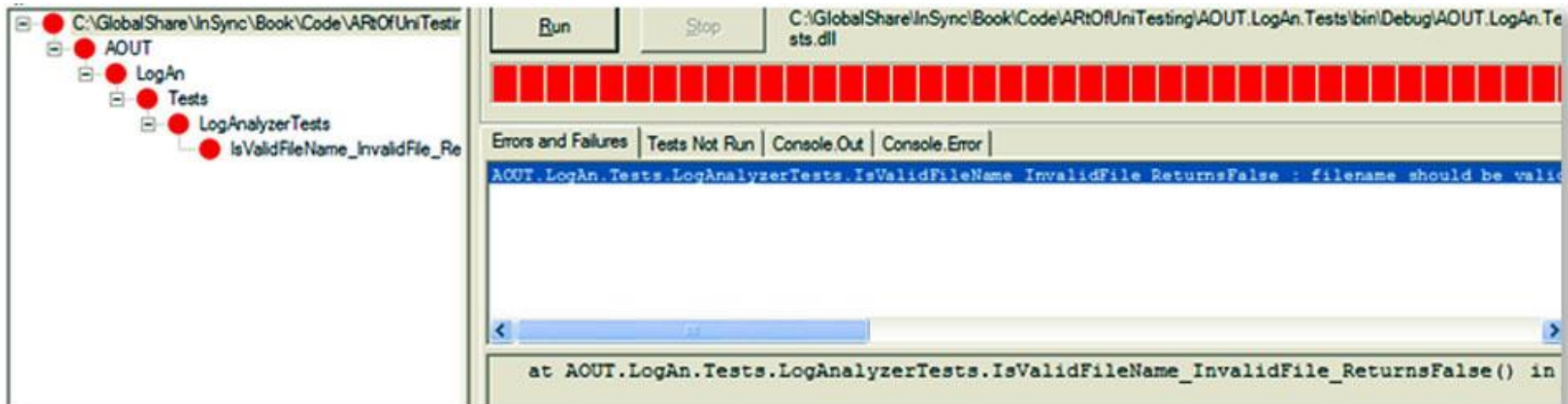
Object to be tested	Object to create on the testing side
Project	Create a test project named [ProjectUnderTest].Tests.
Class	For each class, create at least one class with the name [ClassName]Tests.
Method	For each method, create at least one test method with the following name: [MethodName]_[StateUnderTest]_[ExpectedBehavior].

First Test - Using AAA Approach

```
[Test]
public void IsValidFileName_validFile_ReturnsTrue()
{
    //arrange
    LogAnalyzer analyzer = new LogAnalyzer();

    //act
    bool result = analyzer.IsValidLogFileName("whatever.slf");

    //assert
    Assert.IsTrue(result, "filename should be valid!");
}
```



Checking for Expected *Exceptions*

```
[Test]
[ExpectedException(typeof(ArgumentException),
    ExpectedMessage = "No filename provided!")]
public void IsValidFileName_EmptyFileName_ThrowsException()
{
    m_analyzer.IsValidLogFileName(string.Empty);
}
```

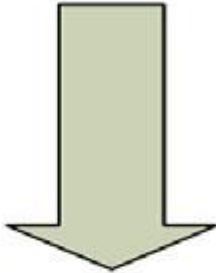


Lab

- **ut-01:** Write your first unit tests in pytest using the AAA pattern, a consistent naming convention, and exception assertions

Setup and Teardown

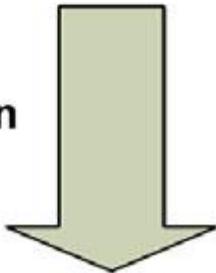
SetUp



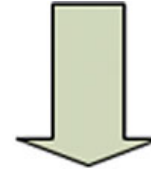
Execute



TearDown



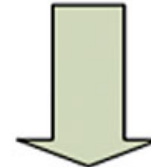
SetUp



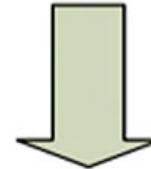
IsValidFileName_validFileLowerCased_ReturnsTrue()



TearDown



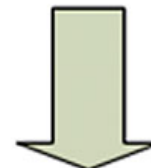
SetUp



IsValidFileName_validFileUpperCased_ReturnsTrue()



TearDown



Lab

- **ut-02:** Eliminate repeated setup code using `setup_method` and `@pytest.fixture`, and understand why test isolation matters

Chapter 3

Using Stubs to Break Dependencies

- *Defining stubs*
- *Refactoring code to use stubs*
- *Overcoming encapsulation problems in code*
- *Exploring best practices when using stubs*



DEFINITION

- An *external dependency* is an object in your system that your code under test interacts with, and over which you have no control. (Common examples are filesystems, threads, memory, time, and so on.)

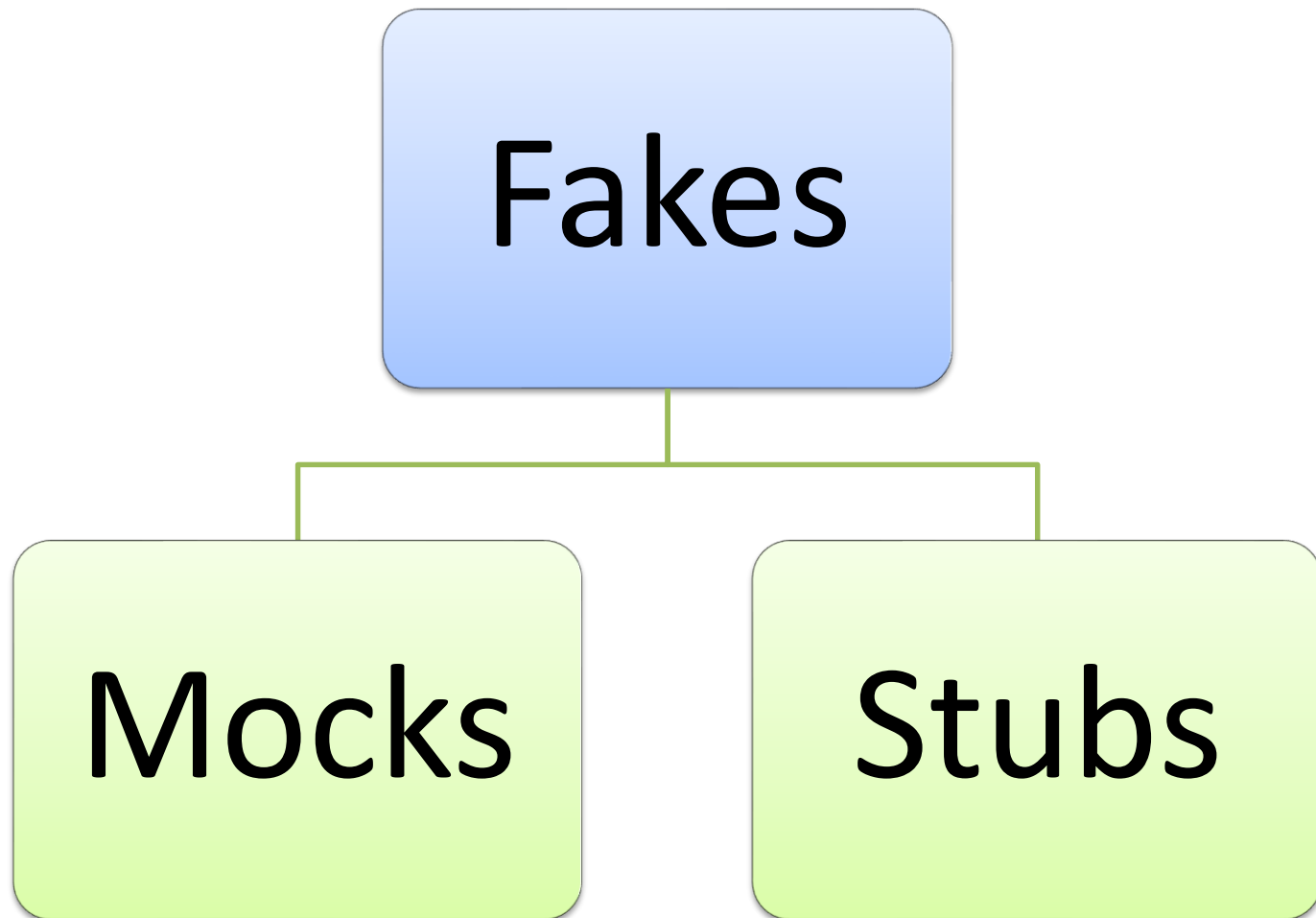


DEFINITION



- A *stub* is a controllable replacement for an existing dependency (or *collaborator*) in the system.
- By using a stub, you can test your code without dealing with the dependency directly.

Basic Unit Test Patterns



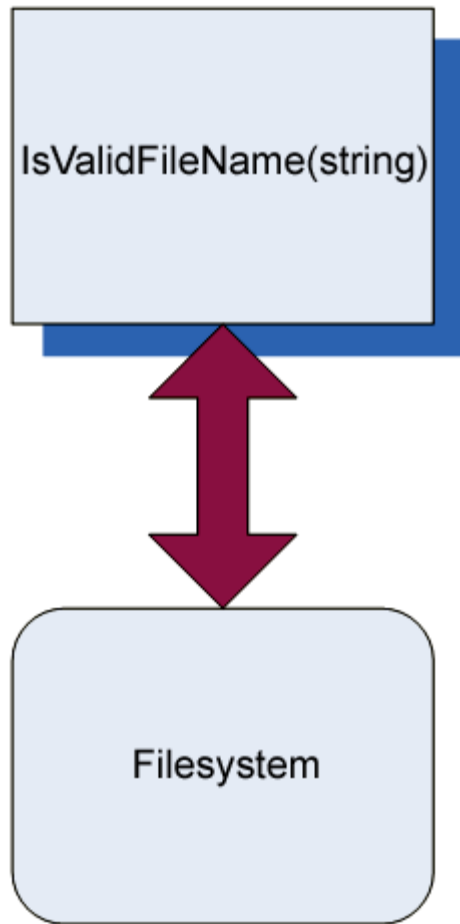
Identifying a Filesystem Dependency in the Application

```
public bool IsValidLogFileName(string fileName)
{
    //read through the configuration file
    //return true if configuration says extension is supported.
}
```



The Problem: An Integration Test

Direct Dependency



test-inhibiting design

the code has some dependency on an external resource, which might break the test even though the code's logic is perfectly valid

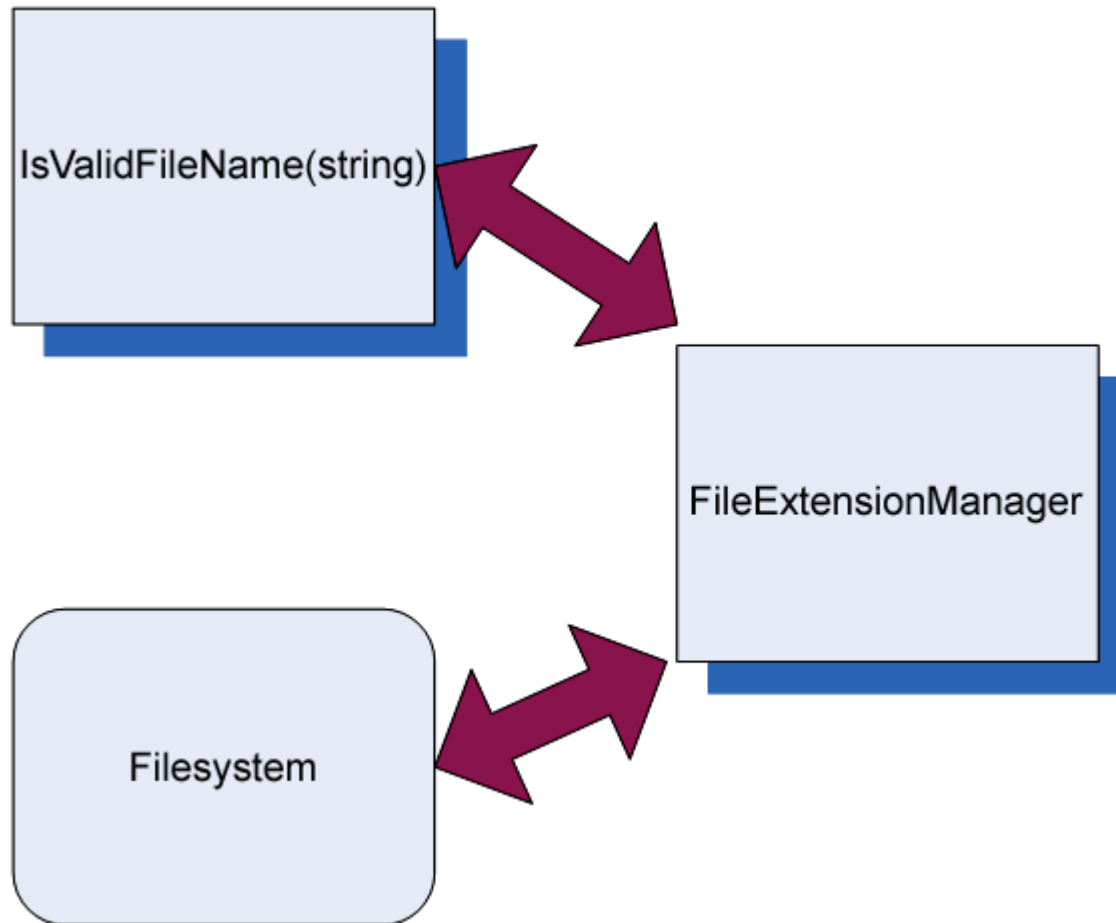
Astronaut Analogy

A Space Shuttle Simulator

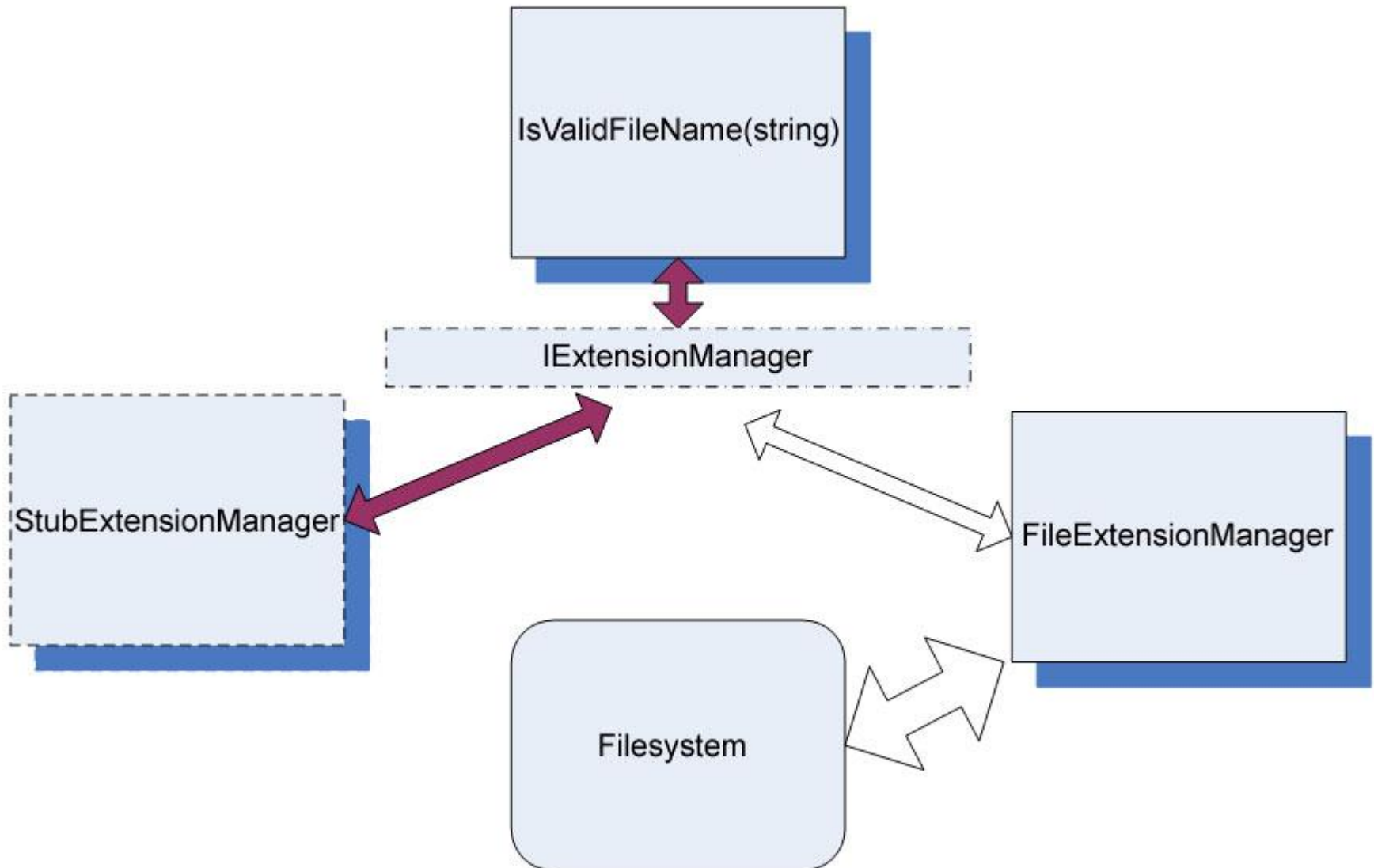


- Find the *interface* or *API*
- Replace the *underlying implementation*

Introducing a Layer of Indirection to Avoid a Direct Dependency



Introducing a Stub to Break the Dependency

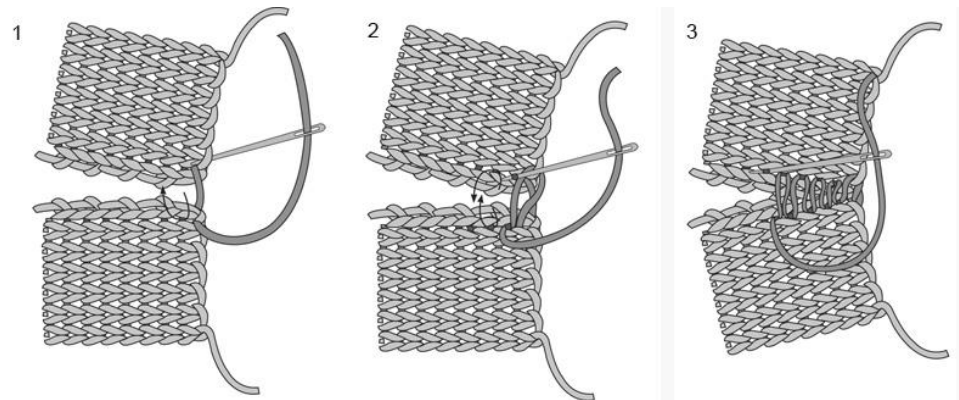


Refactoring the Design to be More Testable

- **DEFINITION** *Refactoring* is the act of changing the code's design without breaking existing functionality.



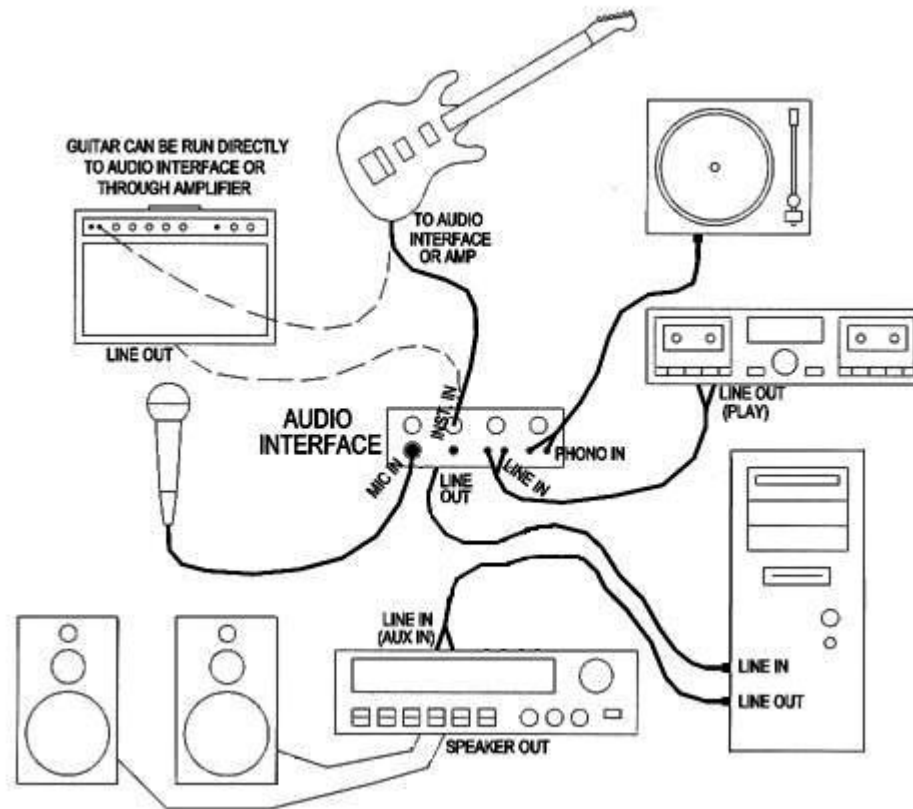
- **DEFINITION** *Seams* are places in your code where you can plug in different functionality, such as stub classes.



Techniques for Breaking Dependencies

- Extract an interface to allow replacing underlying implementation.
- Inject stub implementation into a class under test.
- Receive an interface at the constructor level.
- Receive an interface as a property get or set.
- Get a stub just before a method call.

Extract an *Interface* to Allow Replacing Underlying Implementation



Extracting a *class* that Touches the Filesystem, and Calling it

```
public bool IsValidLogFileName(string fileName)
{
    FileExtensionManager mgr =
        new FileExtensionManager();
    return mgr.IsValid(fileName);
}
```

← Uses the extracted class

```
class FileExtensionManager
{
    public bool IsValid(string fileName)
    {
        //read some file here
    }
}
```

Defines the
extracted class

Extracting an *Interface* from a Known class

```
public class FileExtensionManager : IExtensionManager
{
    public bool IsValid(string fileName)
    {
        ...
    }
}

public interface IExtensionManager
{
    bool IsValid (string fileName);
}
```

← **Implements the interface**

Defines the new interface

```
//the method under test:
public bool IsValidLogFileName(string fileName)
{
    IExtensionManager mgr =
        new FileExtensionManager();
    return mgr.IsValid(fileName);
}
```

← **Defines variable as the type of the interface**

Simple *stub* Code that Always Returns `true`

```
public class StubExtensionManager:IExtensionManager
{
    public bool IsValid(string fileName)
    {
        return true;
    }
}
```

Implements
`IExtensionManager`



A diagram consisting of a horizontal line with an arrow pointing left to the word 'Implements', which is positioned above the text 'IExtensionManager'. A vertical line descends from the horizontal line, and a diagonal line connects the end of the vertical line to the colon in the class declaration 'StubExtensionManager:IExtensionManager'.

Lab

- **ut-03:** Break a hard filesystem dependency by introducing a Protocol and injecting a handwritten stub at the constructor

What's next ???

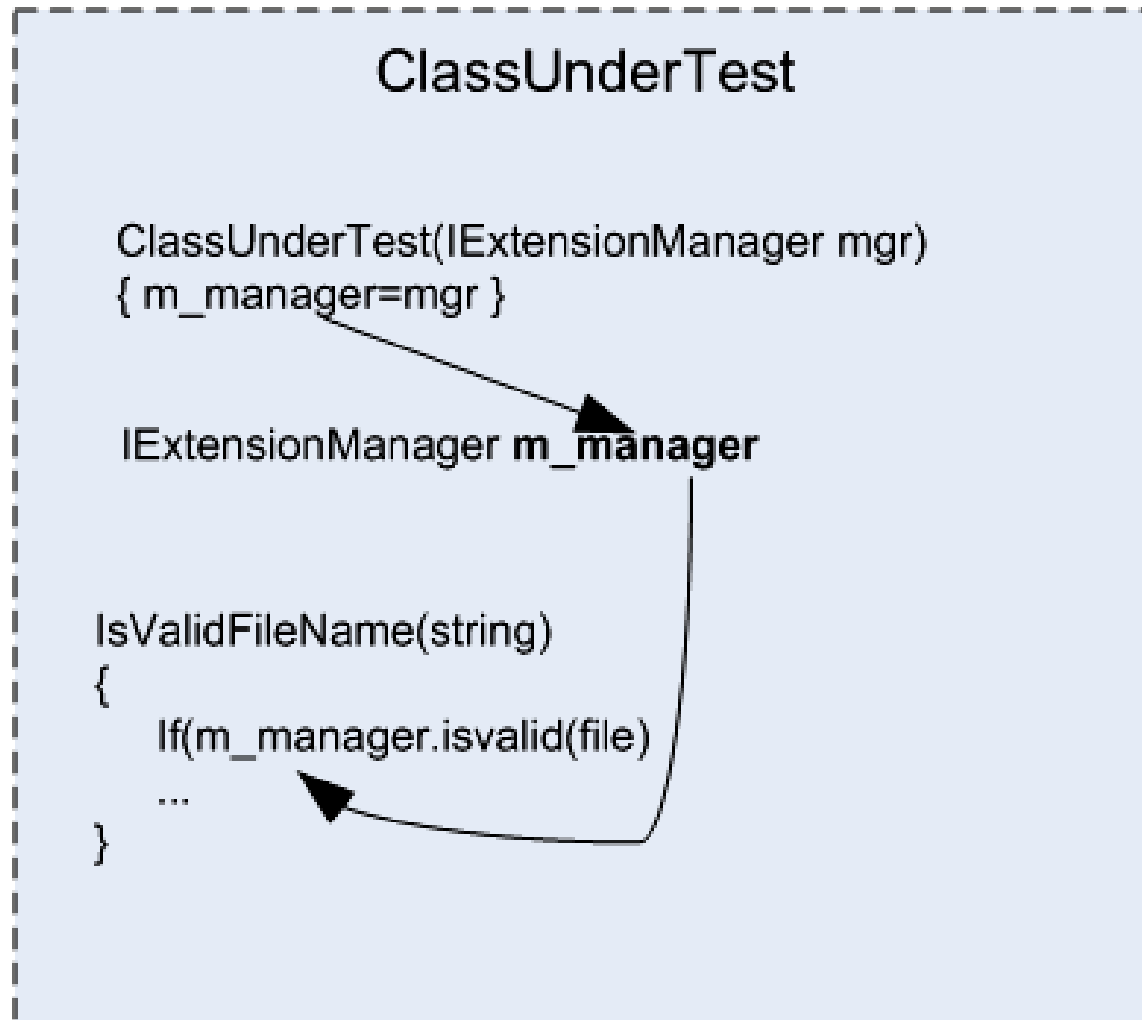
Still have this:

```
public bool IsValidLogFileName(string fileName)
{
    IExtensionManager mgr = new FileExtensionManager();
    return mgr.IsValid (fileName);
}
```

Inject *stub* Implementation into a Class Under Test

- Constructor
- Property
- a parameter to the method *injection*.
- a factory class
- a local factory method

Constructor Injection



Injecting our Stub using Constructor Injection

```
public class LogAnalyzer
{
    private IExtensionManager manager;

    public LogAnalyzer ()
    {
        manager = new FileExtensionManager();
    }
    public LogAnalyzer(IExtensionManager mgr)
    {
        manager = mgr;
    }

    public bool IsValidLogFileName(string fileName)
    {
        return manager.IsValid(fileName);
    }
}
```

← **Defines production code**

← **Creates object in production code**

← **Defines constructor that can be called by tests**



```

public interface IExtensionManager
{
    bool IsValid(string fileName);
}

[TestFixture]                                     ← Defines test code
public class LogAnalyzerTests
{
    [Test]
    public void
IsValidFileName_NameShorterThan6CharsButSupportedExtension_ReturnsFalse()
    {
        StubExtensionManager myFakeManager =
            new StubExtensionManager();
        myFakeManager.ShouldExtensionBeValid
            = true;                                | Sets up stub
                                                | to return true

        //create analyzer and inject stub
        LogAnalyzer log =
            new LogAnalyzer (myFakeManager);      ← Sends in stub

        //Assert logic assuming extension is supported
        bool result = log.IsValidLogFileName("short.ext");
        Assert.IsFalse(result,
            "File name with less than 5 chars should have failed
            the method, even if the extension is supported");
    }
}

```



```
internal class StubExtensionManager : IExtensionManager
{
    public bool ShouldExtensionBeValid;

    public bool IsValid(string fileName)
    {
        return ShouldExtensionBeValid;
    }
}
```



**Defines stub
that uses
simplest
mechanism
possible**

Problems with Constructor Injection:

- More than one stub could be required

```
public LogAnalyzer(IExtensionManager mgr, ILog logger, IWebService  
    service)  
{  
    //    this constructor can be called by tests  
        manager = mgr;  
        log= logger;  
        svc= service;  
}
```



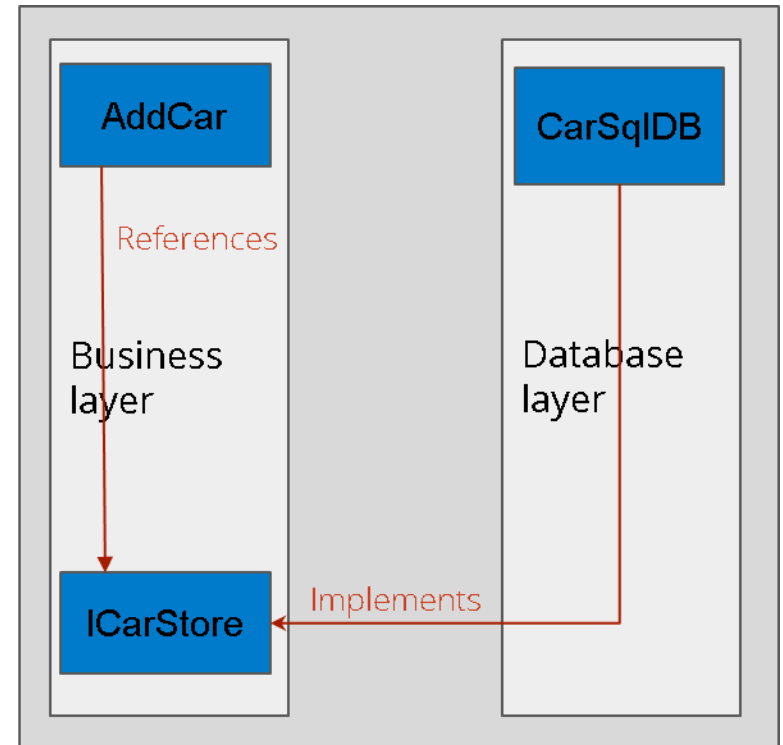
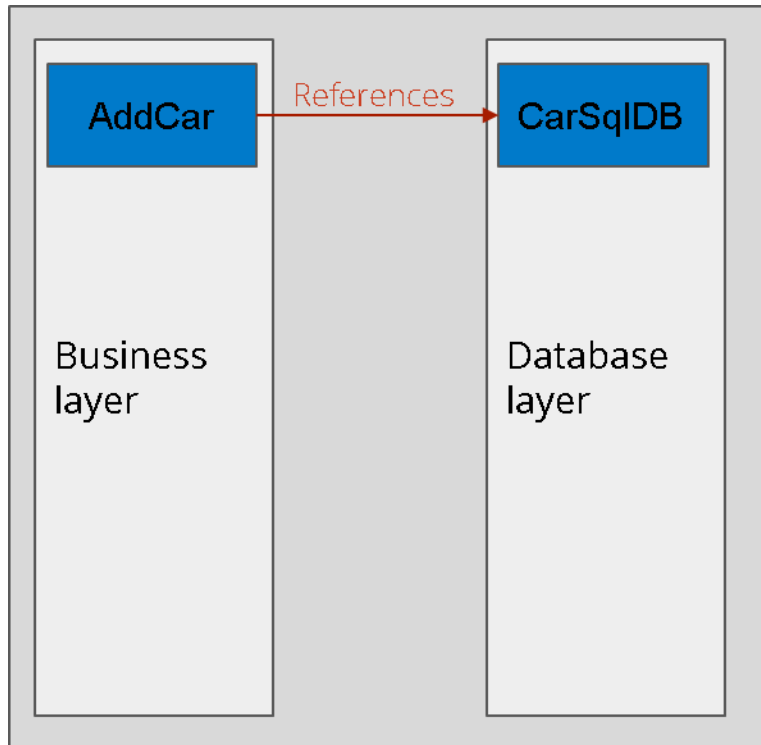
Solution 1:

- *parameter object refactoring*
- create a special class that contains all the values needed to initialize a class



Solution 2:

- Inversion of Control**

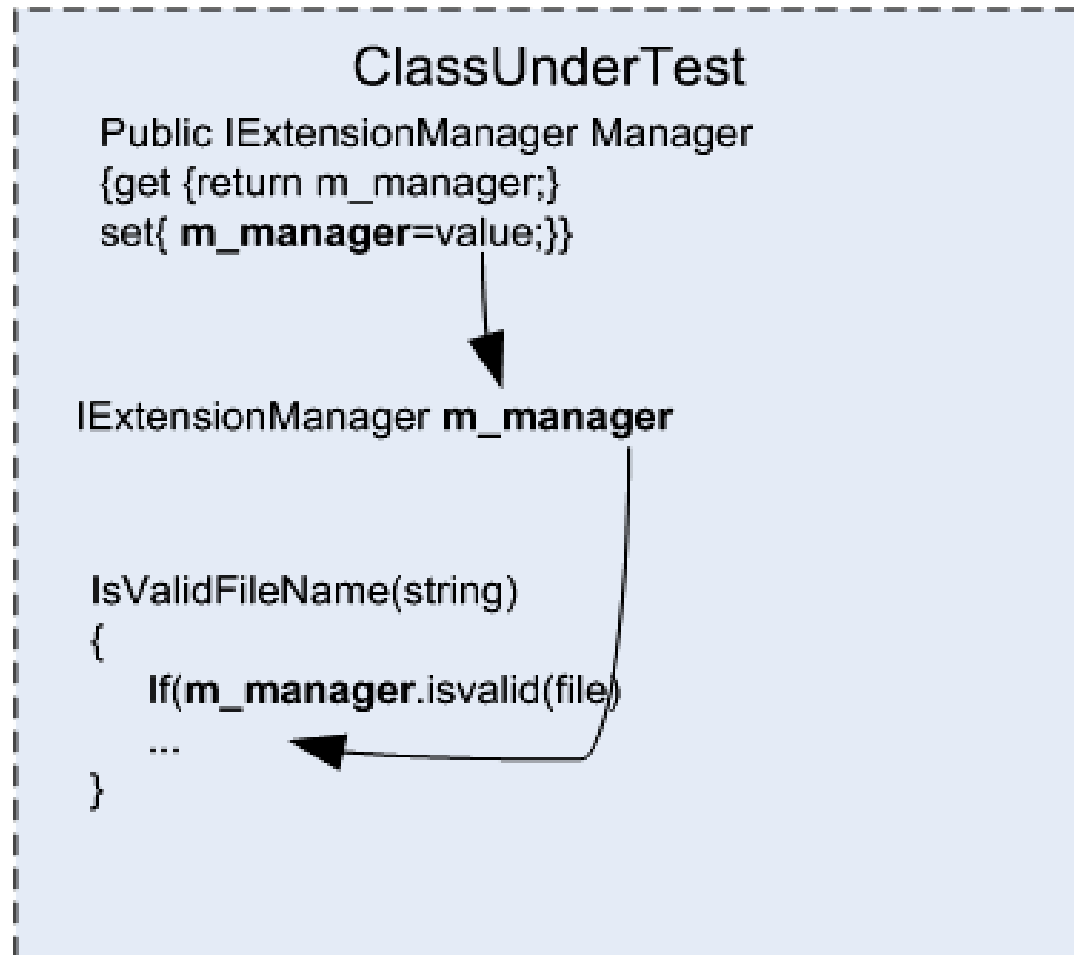


Constructor vs. Parameter Injection



- *Constructor* arguments are **not** optional.
- *Parameters* **are** optional.

Receive an Interface by using *getter* and *setter*



Injecting a *stub* by Adding *setters* to the Class Under Test

```
public class LogAnalyzer
{
    private IExtensionManager manager;

    public LogAnalyzer ()
    {
        manager = new FileExtensionManager();
    }

    public IExtensionManager ExtensionManager
    {
        get { return manager; }
        set { manager = value; }
    }

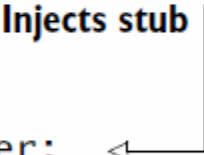
    public bool IsValidLogFileName(string fileName)
    {
        return manager.IsValid(fileName);
    }
}
```

**Allows setting
dependency
via a property**

```
[Test]
    Public void
IsValidFileName_NameShorterThan6CharsButSupportedExtension_ReturnsFalse()
    {
        //set up the stub to use, make sure it returns true
        ...

        //create analyzer and inject stub
        LogAnalyzer log =
            new LogAnalyzer ();
        log.ExtensionManager=someFakeManagerCreatedEarlier;

        //Assert logic assuming extension is supported
        ...
    }
}
```



When to use getter and setter Injection?

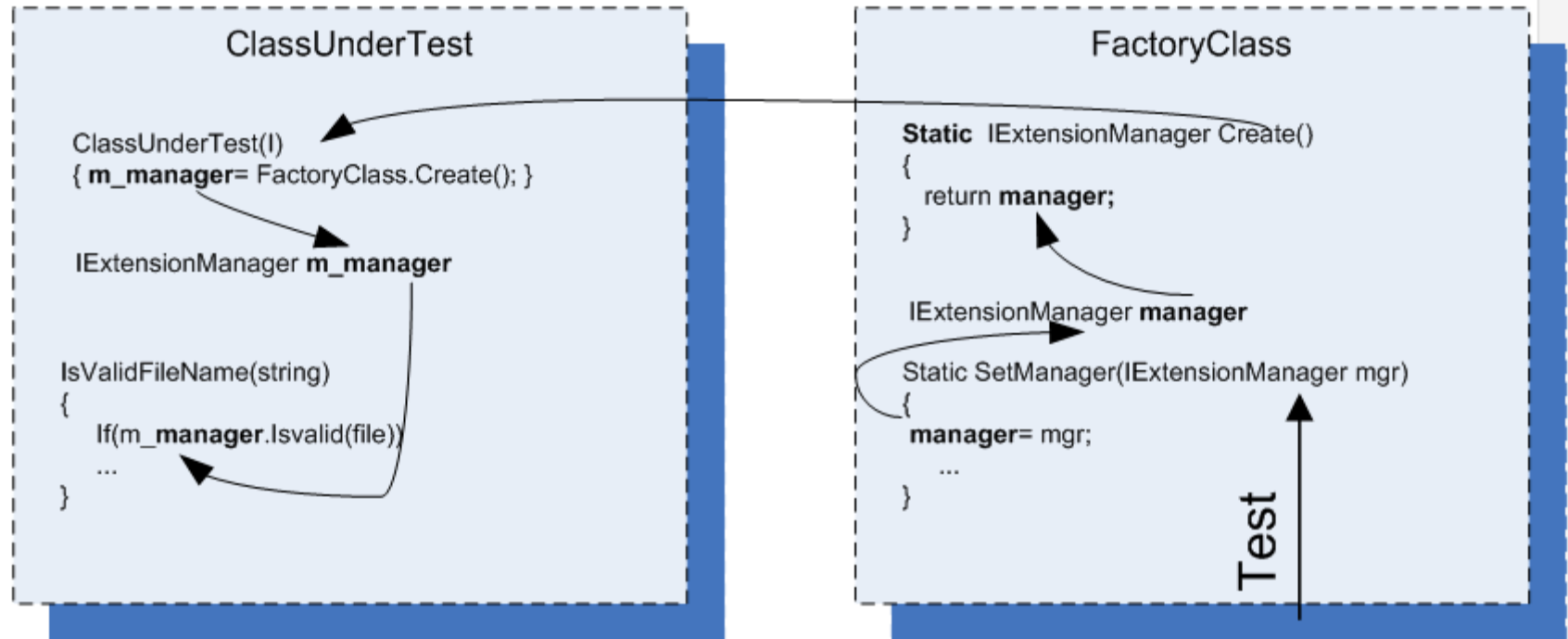
- ✓ When a dependency of the class under test is optional
- ✓ When a dependency has a default instance created



Lab

- **ut-04:** Learn when to use constructor injection for required dependencies and property injection for optional ones

Use a *factory* Class for Complex Objects



Class under test

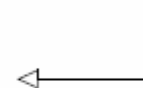


```
public class LogAnalyzer
{
    private IExtensionManager manager;

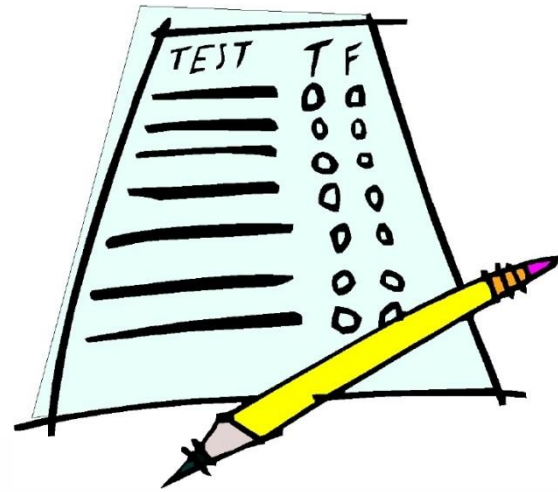
    public LogAnalyzer ()
    {
        manager = ExtensionManagerFactory.Create();
    }

    public bool IsValidLogFileName(string fileName)
    {
        return manager.IsValid(fileName)
            && Path.GetFileNameWithoutExtension(fileName).Length>5;
    }
}
```

**Uses factory in
production code**



Test



```
[Test]
    Public void
IsValidFileName_NameShorterThan6CharsButSupportedExtension_ReturnsFalse()
{
    //set up the stub to use, make sure it returns true
    ...
    ExtensionManagerFactory
        .SetManager(myFakeManager);
    //create analyzer and inject stub
    LogAnalyzer log =
        new LogAnalyzer ();

    //Assert logic assuming extension is supported
    ...
}
}
```

Sets stub into
factory class
for this test

Factory



```
Class ExtensionManagerFactory
{
    Private IExtensionManager customManager=null;
    Public IExtensionManager Create()
    {
        If(customManager!=null)
return customManager;
        Return new FileExtensionManager();
    }
    Public void SetManager(IExtensionManager mgr)
    {
        customManager = mgr;
    }
}
```

**Defines factory that can use
and return custom manager**

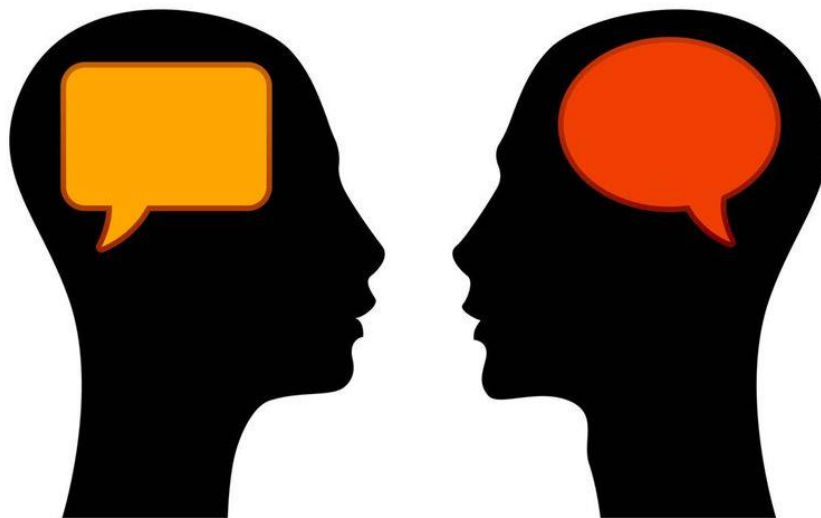
Chapter 4

Interaction Testing using *mock* Objects

- *Defining interaction testing*
- *Understanding mock objects*
- *Differentiating mocks and stubs*
- *Exploring mock object best practices*

DEFINITION

Interaction testing is testing how an object sends input to or receives input from other objects - how that object *interacts* with other objects.

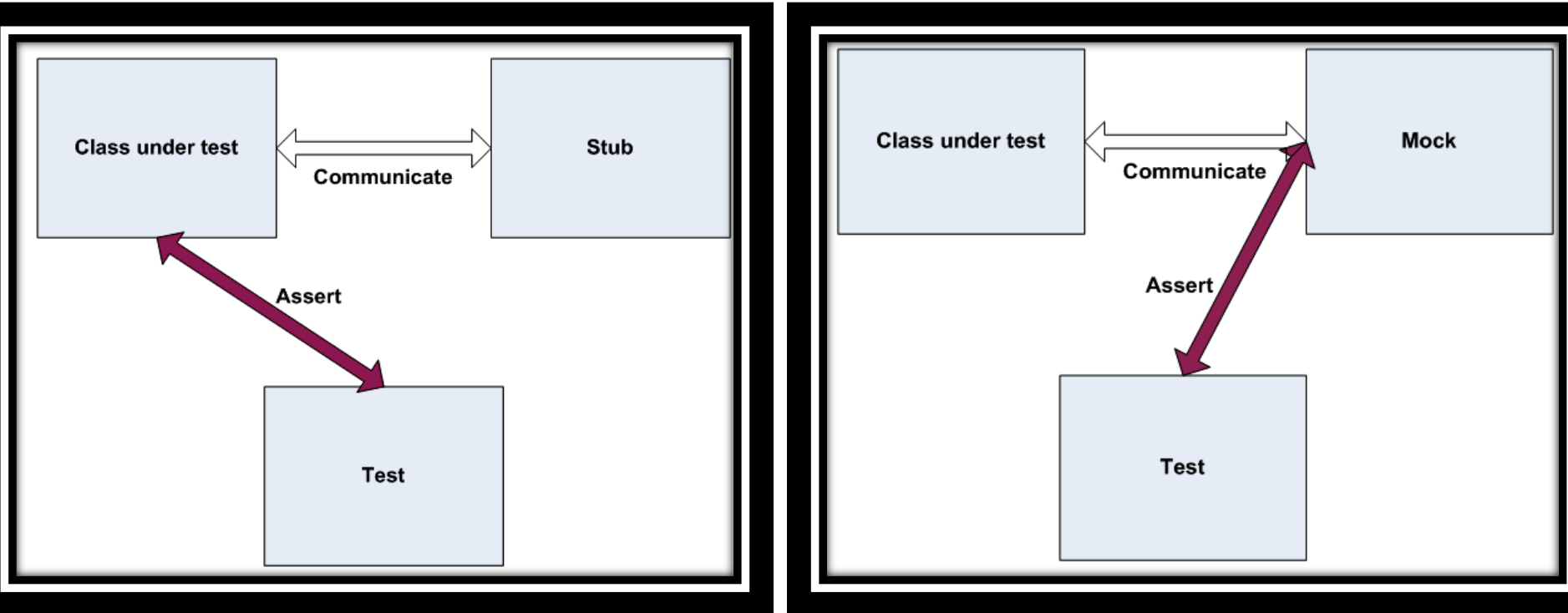


DEFINITION

- A *mock object* is a fake object in the system that decides whether the unit test has passed or failed.
- It does so by verifying whether the object under test interacted as expected with the fake object.
- There's usually **no more than one** mock **per test**.

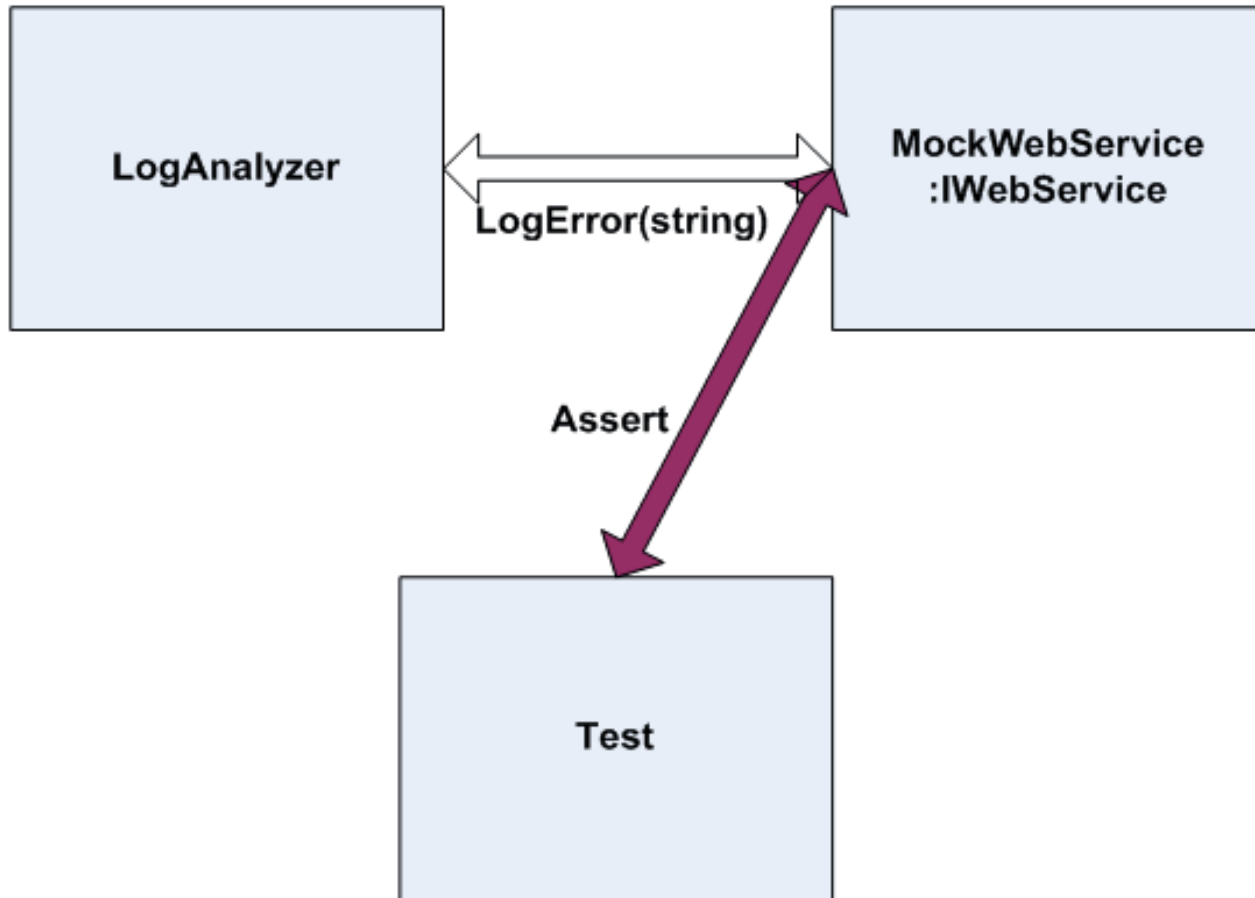


Differences Between *stub* and *mock*



stubs can't fail tests but *mocks* can!

MockWebService to record messages that LogAnalyzer will send



Interface

```
public interface IWebService
{
    void LogError(string message);
}
```

Mock = Stub + ExtraCode

```
public class MockService:IWebService
{
    public string LastError;

    public void LogError(string message)
    {
        LastError = message;
    }
}
```



Lab

- **ut-05:** Understand the difference between stubs and mocks by writing a mock that records calls and verifies behavior

[Test]

```
public void Analyze_TooShortFileName_CallsWebService()
{
    MockService mockService = new MockService();
    LogAnalyzer log = new LogAnalyzer(mockService);
    string tooShortFileName="abc.ext";
    log.Analyze(tooShortFileName);

    Assert.AreEqual("Filename too short:abc.ext",
                    mockService.LastError);
}
```

**Asserts against
mock object**

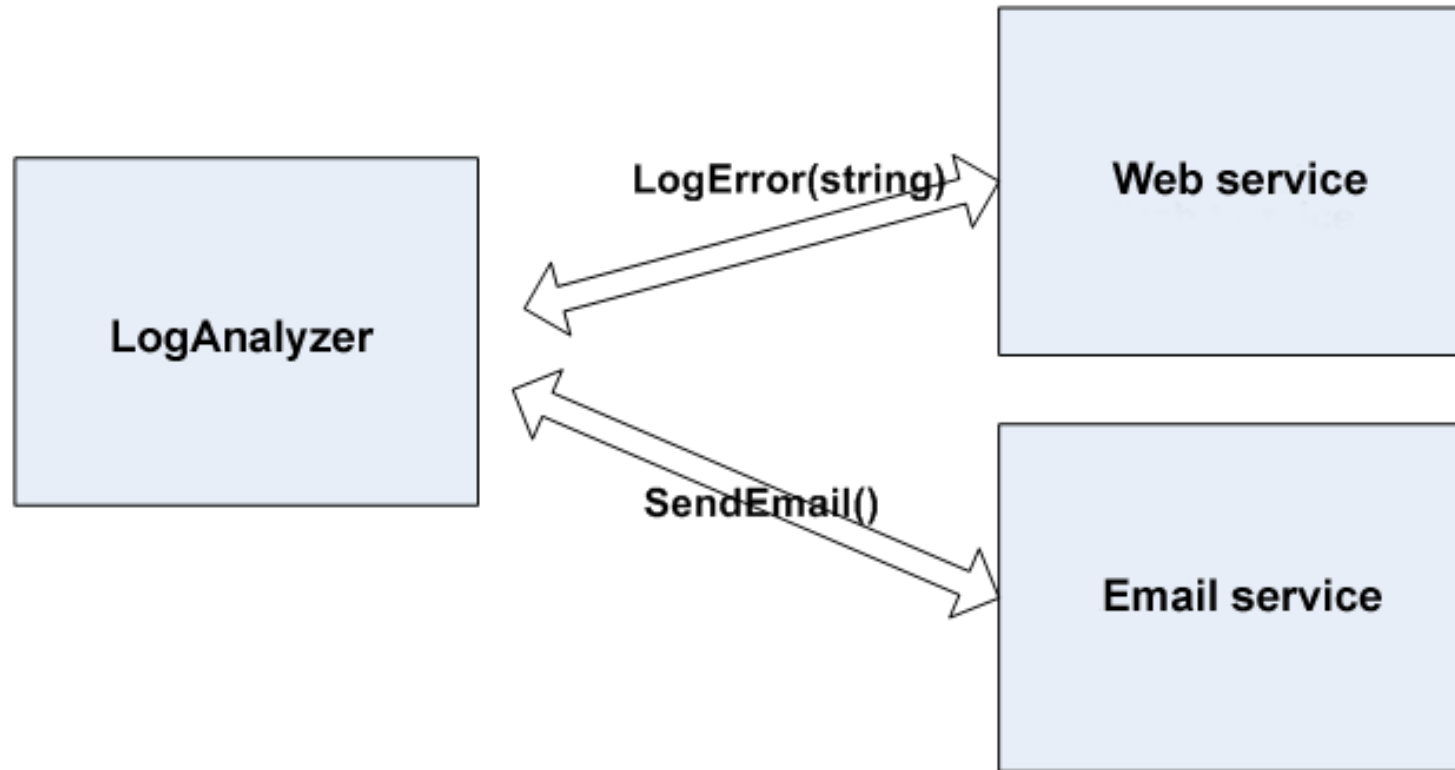
```
public class LogAnalyzer
{
    private IWebService service;

    public LogAnalyzer(IWebService service)
    {
        this.service = service;
    }

    public void Analyze(string fileName)
    {
        if(fileName.Length<8)
        {
            service.LogError("Filename too short:"
                             + fileName);
        }
    }
}
```

**Logs error in
production code**

Stub + Mock Together

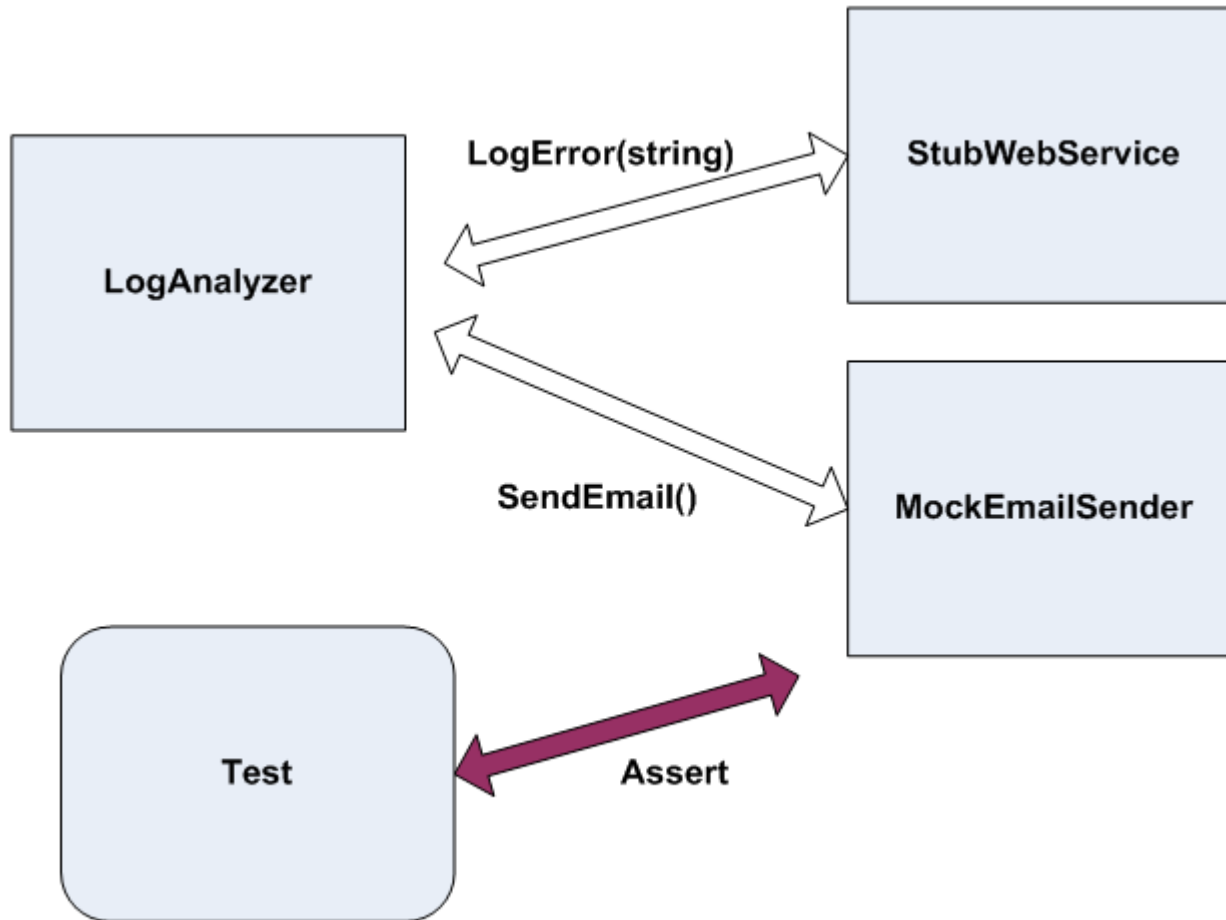


- *Mock* = Email service
- *Stub* = Web Service

E-Mail to Admin when Web-Service Fails

```
if(fileName.Length<8)
{
    try
    {
        service.LogError("Filename too short:" + fileName);
    }
    catch (Exception e)
    {
        email.SendEmail("a","subject",e.Message);
    }
}
```





Email Interface

```
public interface IEmailService
{
    void SendEmail(string to, string subject, string body);
}
```



```
public class LogAnalyzer2
```

```
{
```

```
    private IWebService service;
```

```
    private IEmailService email;
```

```
    public IWebService Service
```

```
    {
```

```
        get { return service; }
```

```
        set { service = value; }
```

```
    }
```

```
    public IEmailService Email
```

```
    {
```

```
        get { return email; }
```

```
        set { email = value; }
```

```
    }
```

```
    public void Analyze(string fileName)
```

```
    {
```

```
        if(fileName.Length<8)
```

```
        {
```

```
            try
```

```
            {
```

```
                service.LogError("Filename too short:" + fileName);
```

```
            }
```

```
            catch (Exception e)
```

```
            {
```

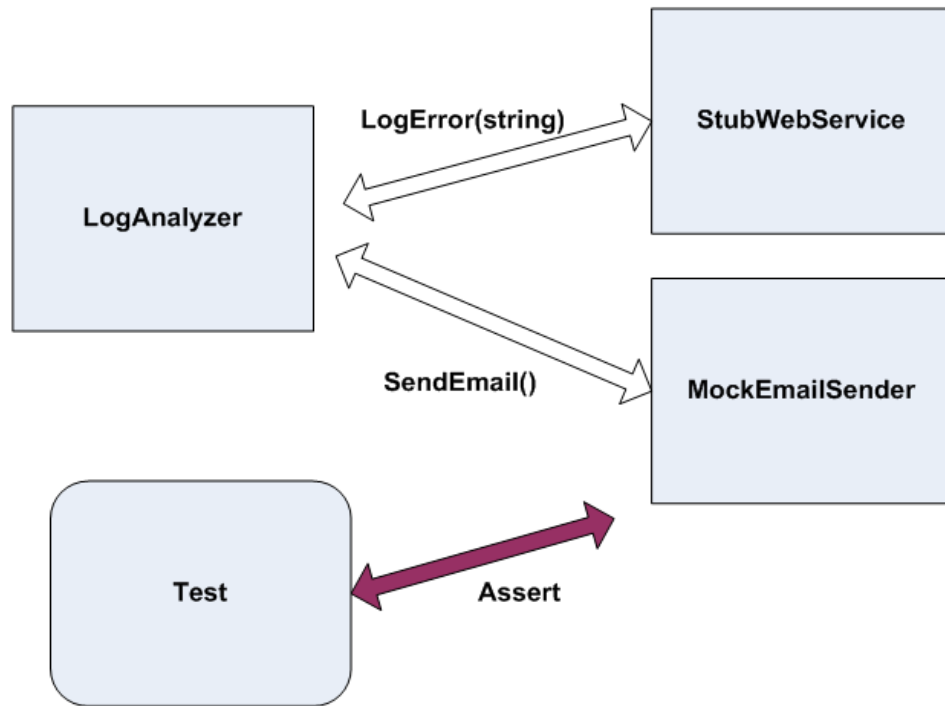
```
                email.SendEmail("a","subject",e.Message);
```

```
            }
```

```
        }
```

```
    }
```

```
}
```



```
[TestFixture]
public class LogAnalyzer2Tests
{
    [Test]
    public void Analyze_WebServiceThrows_SendsEmail()
    {
        StubService stubService = new StubService();
        stubService.ToThrow= new Exception("fake exception");

        MockEmailService mockEmail = new MockEmailService();

        LogAnalyzer2 log = new LogAnalyzer2();
        //we use setters instead of
        //constructor parameters for easier coding
        log.Service = stubService
        log.Email=mockEmail;

        string tooShortFileName="abc.ext";
        log.Analyze(tooShortFileName);

        Assert.AreEqual("a",mockEmail.To);
        Assert.AreEqual("fake exception",mockEmail.Body);
        Assert.AreEqual("subject",mockEmail.Subject);
    }
}
```

← ? Using properties instead
of constructor injection

Stub - WebService

```
public class StubService:IWebService
{
    public Exception ToThrow;
    public void LogError(string message)
    {
        if(ToThrow!=null)
        {
            throw ToThrow;
        }
    }
}
```

Mock - Email Service

```
public class MockEmailService:IEmailService
{
    public string To;
    public string Subject;
    public string Body;

    public void SendEmail(string to,
                           string subject,
                           string body)
    {
        To = to;
        Subject = subject;
        Body = body;
    }
}
```

RULE:

- ✓ One *mock* per test
- ✓ Other fakes = *stubs*



Lab

- **ut-06:** Combine a stub and a mock in the same test to verify both control flow and expected side effects

Chapter 5

Isolation (mock object) Frameworks

- Understanding isolation frameworks
- Defining fake objects
- Using **Pytest** Mocks to create stubs and mocks
- Surveying advanced use cases for mocks and stubs
- Exploring arrange-act-assert and record-and-replay syntax
- Avoiding common misuses of isolation frameworks

Why using Isolation Frameworks?

DEFINITION

- An isolation framework is a set of programmable APIs that make creating mock and stub objects much easier.
- Isolation frameworks save the developer from the need to write repetitive code to test or simulate object interactions.

A problem:

Complicated interface

```
public interface IComplicatedInterface
{
    void Method1(string a, string b, bool c, int x, object o);
    void Method2(string b, bool c, int x, object o);
    void Method3(bool c, int x, object o);
}
```

```

class MytestableComplicatedInterface: IComplicatedInterface
{
    public string meth1_a;
    public string meth1_b, meth2_b;
    public bool meth1_c, meth2_c, meth3_c;
    public int meth1_x, meth2_x, meth3_x;
    public int meth1_0, meth2_0, meth3_0;

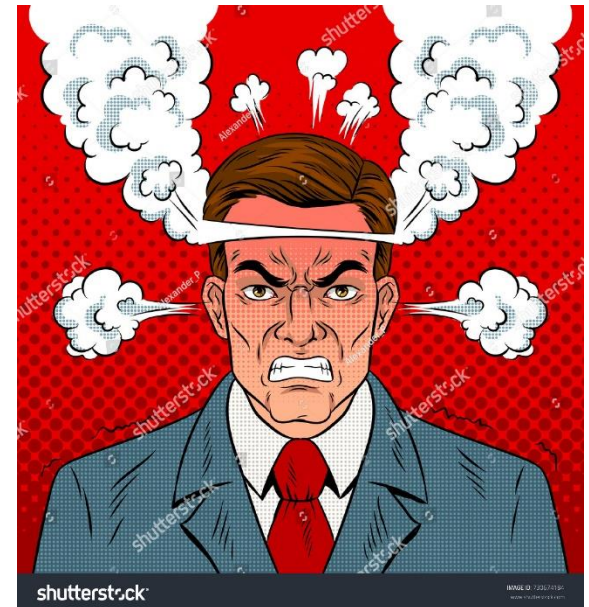
    public void Method1(string a,
string b, bool c,
int x, object o)
    {
        meth1_a = a;
        meth1_b = b;
        meth1_c = c;
        meth1_x = x;
        meth1_0 = 0;
    }

    public void Method2(string b, bool c, int x, object o)
    {
        meth2_b = b;
        meth2_c = c;
        meth2_x = x;
        meth2_0 = 0;
    }

    public void Method3(bool c, int x, object o)
    {
        meth3_c = c;
        meth3_x = x;
        meth3_0 = 0;
    }
}

```

Handwritten stubs



Dynamically Creating a Fake Object

DEFINITION

A *dynamic fake object* is any stub or mock that's created at runtime without needing to use a handwritten implementation of an interface or subclass.

Dynamic Mock

[Test]

```
public void Analyze_TooShortFileName_ErrorLoggedToService()
{
    MockRepository mocks = new MockRepository();
    IWebService simulatedService =
    mocks.StrictMock<IWebService>();
```

? Creates dynamic
mock object

```
    using(mocks.Record())
    {
        simulatedService
        .LogError("Filename too short:abc.ext");
    }
```

W Sets
expectation

```
        LogAnalyzer log =
            new LogAnalyzer(simulatedService);
        string tooShortFileName="abc.ext";
        log.Analyze(tooShortFileName);
```

4 Invokes
LogAnalyzer

```
    mocks.Verify(simulatedService);
}
```

? Asserts expectations
have been met

Lab

- **ut-07:** Replace handwritten fakes with MagicMock and mocker.patch to see where isolation frameworks save effort

Strict Mock can Fail in Two Ways:

- expected methods aren't called on it
- an unexpected method is called on it

Non-strict Mock can Fail in One Way:

- expected methods aren't called on it

Non-strict Mock Example

```
[Test]
public void Analyze_TooShortFileName_ErrorLoggedToService()
{
    MockRepository mocks = new MockRepository();
    IWebService simulatedService =
        MockRepository.DynamicMock<IWebService>();

    using(mocks.Record())
    {
        //we expected "Filename too short:abc.ext"
        simulatedService.LogError("bad string");
    }

    LogAnalyzer log = new LogAnalyzer(simulatedService);
    string tooShortFileName="abc.ext";
    log.Analyze(tooShortFileName);

    mocks.VerifyAll();
}
```

Lab

- **ut-08:** See how a non-strict mock silently swallows typos, and when to enforce `spec=` or `create_autospec` instead

Returning Values from Fake Objects

[Test]

```
public void ReturnResultsFromMock()  
{
```

```
    MockRepository mocks = new MockRepository();  
    IGetResults resultGetter = mocks.DynamicMock<IGetResults>();  
    using(mocks.Record())  
    {
```

```
        resultGetter.GetSomeNumber("a");  
        LastCall.Return(1);
```

```
        resultGetter.GetSomeNumber("a");  
        LastCall.Return(2);
```

```
        resultGetter.GetSomeNumber("b");  
        LastCall.Return(3);
```

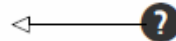
```
    }
```

```
    int result = resultGetter.GetSomeNumber("b");  
    Assert.AreEqual(3, result);
```

```
    int result2 = resultGetter.GetSomeNumber("a");  
    Assert.AreEqual(1, result2);
```

```
    int result3 = resultGetter.GetSomeNumber("a");  
    Assert.AreEqual(2, result3);
```

```
}
```



Forces method call
to return value

**Order
matters**

**Order does
not matter**

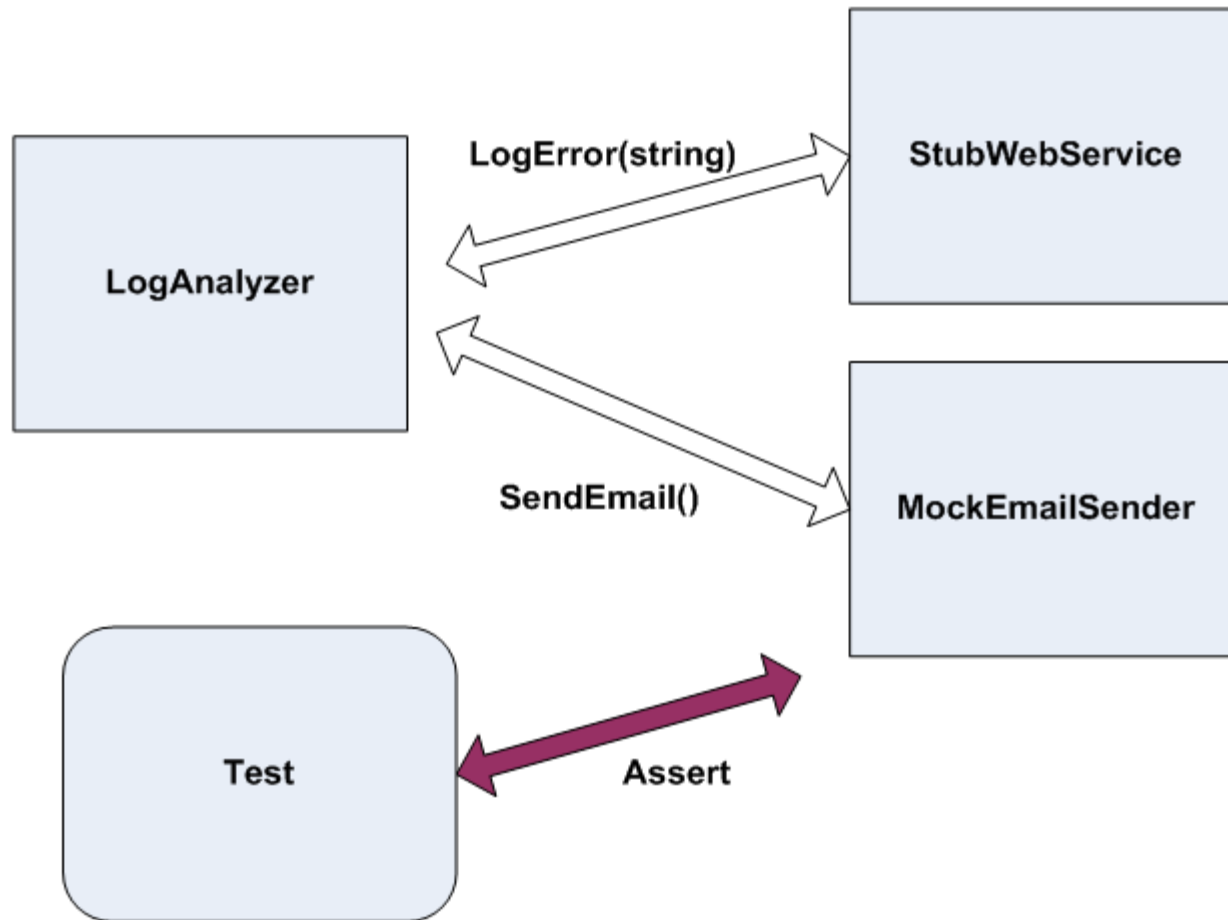
Test doesn't Call an Expected Method

```
[Test]
public void StubNeverFailsTheTest()
{
    MockRepository mocks = new MockRepository();
    IGetResults resultGetter = mocks.Stub<IGetResults>();
    using(mocks.Record())
    {
        resultGetter.GetSomeNumber("a");
        LastCall.Return(1);
    }
    resultGetter.GetSomeNumber("b");
    mocks.Verify(resultGetter);
}
```

Specifies a rule,
not an expectation

← Will never fail on stubs

Combining Dynamic *stubs* and *mocks*



With Dynamic *mocks* & *stubs*

```
[Test]
public void Analyze_WebServiceThrows_SendsEmail()
{
    MockRepository mocks = new MockRepository();
    IWebService stubService =
        mocks.Stub<IWebService>();
    IEmailService mockEmail =
        mocks.StrictMock<IEmailService>();

    using(mocks.Record())
    {
        stubService.LogError("whatever");
        LastCall.Constraints(Is.Anything());
        LastCall.Throw(new Exception("fake exception"));

        mockEmail.SendEmail("a", "subject", "fake exception");
    }

    LogAnalyzer2 log = new LogAnalyzer2();
    log.Service = stubService;
    log.Email = mockEmail;

    string tooShortFileName = "abc.ext";
    log.Analyze(tooShortFileName);

    mocks.VerifyAll();
}
```

← ? Ensures exception is thrown

